

Operating Systems and System Programming

Skill-4

Task-1:

Aim

To implement a simple command-line tokenizer and parser that splits user input into tokens, handles whitespace and delimiters, validates the input, detects syntax errors and empty commands, and produces a basic execution structure.

Algorithm

1. Start the program.
2. Read the command from the user.
3. Check whether the input is empty.
4. Identify delimiters such as spaces and tabs.
5. Split the input into individual tokens.
6. Store and display the tokens.
7. Identify the command and its arguments.
8. Validate the command syntax.
9. Detect invalid or empty input.
10. Display the parsed/execution structure.
11. Stop the program.

Task-1: Tokenization

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill4.c *  
#include <stdio.h>  
#include <string.h>  
  
int main()  
{  
    char input[100];  
    char *token;  
    int count = 0;  
  
    printf("Enter command: ");  
    fgets(input, sizeof(input), stdin);  
  
    token = strtok(input, " \\t\\n");  
  
    if (token == NULL)  
    {  
        printf("Empty command\\n");  
        return 0;  
    }  
  
    printf("\\nTokens:\\n");  
  
    while (token != NULL)
```

```
    printf("\\nTokens:\\n");  
  
    while (token != NULL)  
    {  
        printf("Token %d: %s\\n", ++count, token);  
        token = strtok(NULL, " \\t\\n");  
    }  
  
    printf("\\nTotal tokens: %d\\n", count);  
  
    return 0;  
}
```

Compile and Run

```
allur@AKSHAYA: ~$ gcc skill4.c -o skill4
```

```
allur@AKSHAYA:~$ ./skill4
Enter command: ls -l file.txt

Tokens:
Token 1: ls
Token 2: -l
Token 3: file.txt

Total tokens: 3
allur@AKSHAYA:~$ |
```

Task-2: Parser

```
allur@AKSHAYA: ~  x  +  v
GNU nano 8.7.1 skill4-2.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];
    char *token;
    int count = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    token = strtok(input, " \\t\\n");

    if (token == NULL)
    {
        printf("Error: Empty command\\n");
        return 0;
    }

    printf("\\nExecution Structure:\\n");
    printf("Command: %s\\n", token);

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C
    ^X Exit      ^R Read File  ^\\ Replace    ^U Paste      ^J Justify    ^/
```

```

printf("\nExecution Structure:\n");
printf("Command: %s\n", token);

token = strtok(NULL, " \t\n");

while (token != NULL)
{
    printf("Argument %d: %s\n", ++count, token);
    token = strtok(NULL, " \t\n");
}

if (count == 0)
    printf("No arguments\n");

printf("\nSyntax validation: Successful\n");

return 0;
}

```

^G Help
^O Write Out
^F Where Is
^K Cut
^T Execute
^X Exit
^R Read File
^_ Replace
^U Paste
^J Justify

Compile and Run

```

allur@AKSHAYA:~$ gcc skill4-2.c -o skill4-2
allur@AKSHAYA:~$ ./skill4-2
Enter command: ls -l file.txt

Execution Structure:
Command: ls
Argument 1: -l
Argument 2: file.txt

Syntax validation: Successful
allur@AKSHAYA:~$ ./skill4-2
Enter command:
Error: Empty command
allur@AKSHAYA:~$

```

Result

The command input was successfully tokenized and parsed. Whitespace and delimiters were handled correctly, tokens and arguments were identified, empty commands were detected, and the resulting execution structure was displayed successfully.